



Deploy, xavfsizlik va testlash

Ilovani internetga chiqarish (PaaS, VPS, Docker, Nginx, HTTPS), CI/CD, OWASP asosiy zaifliklari (XSS, SQLi, CSRF), unumdorlik hamda avtomatik testlar (node:test, Supertest, Vitest).

Yuqori · ~150 daqiqa

Kompyuteringizda ishlaydigan ilova hali mahsulot emas. Uni haqiqiy foydalanuvchilarga yetkazish uchun: (1) internetdagi serverga **joylash (deploy)**, (2) hujumlardan **himoyalash**, (3) har bir o'zgarishdan keyin hech narsa buzilmaganini **avtomatik tekshirish** kerak. Bu dars full-stack dasturchini "kod yozuvchi"dan "mahsulot yetkazuvchi"ga aylantiradi.

Deploy: muhitlar va asosiy tushunchalar

- **Development** — sizning kompyuteringiz: batafsil xatolar, avtomatik qayta yuklash.
- **Staging** — production'ning nusxasi, sinov uchun.
- **Production** — haqiqiy foydalanuvchilar: minifikatsiya, kesh, xatolar yashirilgan, HTTPS majburiy.

12-Factor App tamoyillarining eng muhimlari: konfiguratsiya muhit o'zgaruvchilarida; ilova holatsiz (fayllar va sessiyalar diskda emas — baza/S3/Redis'da); loglar `stdout` ga; build va ishga tushirish bosqichlari ajratilgan; dev va prod iloji boricha o'xshash.

Hosting turlari

Tur	Misollar	Afzalligi	Kamchiligi	Qachon
Statik hosting	GitHub Pages, Netlify, Vercel, Cloudflare Pages	Bepul, tez (CDN), sozlash shart emas	Faqat frontend	Portfolio, landing, React SPA

PaaS	Render, Railway, Fly.io, Heroku	Git push → deploy; baza, HTTPS tayyor	Qimmatroq, kamroq nazorat	Birinchi loyihalar, startaplar
VPS	DigitalOcean, Hetzner, AWS EC2, mahalliy provayderlar	To'liq nazorat, arzon	Hammasini o'zingiz sozlaysiz va himoyalaysiz	O'rganish, maxsus talablar
Konteyner platformalar	AWS ECS, Google Cloud Run, Kubernetes	Masshtablash	Murakkab	Katta tizimlar

Docker asoslari

Docker ilovani barcha bog'liqliklari (Node versiyasi, paketlar, tizim kutubxonalari) bilan birga **konteynerga** o'raydi. "Mening kompyuterimda ishlayapti" muammosi yo'qoladi: konteyner har qayerda bir xil ishlaydi.

- **Image** — ilovaning "qolipi" (faqat o'qish uchun), `Dockerfile` asosida quriladi.
- **Container** — image'ning ishlab turgan nusxasi.
- **Volume** — konteyner o'chsa ham saqlanadigan ma'lumot (baza fayllari).
- **Docker Compose** — bir nechta konteynerni (API + baza) bitta faylda tasvirlash.

Terminal

```
docker build -t notes-app .
docker run -d -p 3000:3000 --env-file .env --name notes notes-app
docker ps                # ishlayotgan konteynerlar
docker logs -f notes     # loglar
docker exec -it notes sh  # konteyner ichiga kirish
docker stop notes && docker rm notes
```

Nginx, PM2 va HTTPS (VPS uchun)

VPS'da odatda Node.js ilovasi to'g'ridan-to'g'ri 80/443-portga qo'yilmaydi. Uning oldida **Nginx** turadi — **teskari proksi (reverse proxy)**:



/etc/nginx/sites-available/notes

```
server {
    server_name notes.example.uz;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade;           # WebSocket uchun
        proxy_set_header Connection "upgrade";
    }

    client_max_body_size 5m;
    gzip on;
    gzip_types text/css application/javascript application/json image/svg+xml;
}
```

Terminal

```
# PM2 — Node jarayonlarini boshqaruvchi: yiqilsa qayta ishga tushiradi, server  
qayta yuklansa ham  
npm install -g pm2  
pm2 start src/server.js --name notes -i max      # CPU yadrolari soniga teng  
nusxa (cluster)  
pm2 save && pm2 startup                          # tizim ishga tushganda  
avtomatik  
pm2 logs notes  
pm2 reload notes                                # to'xtamasdan (zero-downtime)  
qayta yuklash  
  
# Bepul HTTPS sertifikat (Let's Encrypt) — avtomatik yangilanadi  
sudo apt install certbot python3-certbot-nginx  
sudo certbot --nginx -d notes.example.uz
```



VPS xavfsizligi minimum

SSH'da parol bilan kirishni o'chiring (faqat kalit), root bilan kirishni taqiqlang, firewall (`ufw allow OpenSSH; ufw allow 'Nginx Full'; ufw enable`), avtomatik xavfsizlik yangilanishlari, PostgreSQL portini (5432) tashqariga ochmang.

CI/CD — avtomatik tekshirish va deploy

CI (Continuous Integration) — har bir push/PR'da testlar va linter avtomatik ishlaydi. **CD (Continuous Delivery/Deployment)** — `main` ga merge qilingan kod avtomatik deploy qilinadi. GitHub'da buning uchun **GitHub Actions** bor.

.github/workflows/ci.yml

```
name: CI
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres: # testlar uchun haqiqiy baza
        image: postgres:17
        env:
          POSTGRES_PASSWORD: test
          POSTGRES_DB: notes_test
        ports: ['5432:5432']
        options: >-
          --health-cmd pg_isready --health-interval 5s --health-timeout 5s --
health-retries 10
        env:
          DATABASE_URL: postgres://postgres:test@localhost:5432/notes_test
          JWT_SECRET: test-secret-test-secret-test-secret
          NODE_ENV: test
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
          cache-dependency-path: server/package-lock.json
      - run: npm ci
        working-directory: server
      - run: psql "$DATABASE_URL" -f db/schema.sql
        working-directory: server
      - run: npm test
        working-directory: server
```

GitHub'da **Settings** → **Branches** → **Branch protection** orqali "CI yashil bo'lmasa merge qilib bo'lmaydi" qoidasini yoqing. Render/Railway kabi platformalar esa `main` yangilanganda o'zi avtomatik deploy qiladi.

Veb-xavfsizlik: OWASP Top 10 asoslari

OWASP — veb-xavfsizlik bo'yicha xalqaro tashkilot; uning "Top 10" ro'yxati eng xavfli zaifliklarni sanab o'tadi. Ularning ko'pchiligini oldingi darslarda uchratdik — endi tizimlashtiramiz.

Zaiflik	Mohiyati	Himoya	Dars
Broken Access Control (IDOR)	Boshqa birovning ma'lumotiga kirish	Har so'rovda egalik/ruxsat tekshiruvi	11, 13
Injection (SQLi)	Ma'lumot kod sifatida bajariladi	Parametrlangan so'rovlar, ORM	10
XSS	Hujumchi skripti boshqa foydalanuvchi brauzerida	<code>textContent</code> , React ekranlashi, DOMPurify, CSP	5, 12
CSRF	Boshqa sayt sizning cookie'ingiz bilan so'rov yuboradi	<code>SameSite</code> cookie, CSRF token, GET'da o'zgartirmaslik	11
Cryptographic Failures	Ochiq parollar, HTTP	bcrypt/argon2, HTTPS, maxfiy kalitlar <code>.env</code> da	8, 11
Security Misconfiguration	Stack trace'lar ko'rinadi, standart parollar, ochiq portlar	Production'da xatolarni yashirish, <code>helmet</code> , firewall	9
Vulnerable Components	Eskirgan paketlardagi ma'lum zaifliklar	<code>npm audit</code> , Dependabot	8
Authentication Failures	Brute-force, zaif parollar	Rate limit, 2FA, umumiy xato xabarlari	11
SSRF	Server foydalanuvchi bergan URL'ga so'rov yuboradi (ichki tarmoqqa)	URL'larni whitelist bilan cheklash	—

XSS chuqurroq va Content Security Policy

JavaScript

```
// Hujum: izoh sifatida yuboriladi
const comment = `>`;

element.innerHTML = comment; // ❌ skript bajariladi
element.textContent = comment; // ✅ matn sifatida ko'rinadi
<p>{comment}</p> // ✅ React avtomatik ekranlaydi
<div dangerouslySetInnerHTML={{ __html: comment }} /> // ❌
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(markdownHtml) }} />
// ✅ tozalangan

// Yana bir yashirin XSS: javascript: havolalar
<a href={user.website}>Sayt</a> // website = "javascript:alert(1)" bo'lsa xavfli!
// → URL'ni tekshiring: faqat http(s):// bilan boshlanganlarga ruxsat
```

CSP (Content-Security-Policy) sarlavhasi — "zaxira himoya": XSS sodir bo'lsa ham, brauzer faqat ruxsat etilgan manbalardagi skriptlarni bajaradi. `helmet()` uni standart sozlamalar bilan qo'yadi:

JavaScript

```
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"], // inline <script> va boshqa
      // domenlar bloklanadi
      imgSrc: ["'self'", 'data:', 'https://cdn.example.uz'],
      connectSrc: ["'self'"],
    },
  },
}));
```

CSRF

Siz `bank.uz` ga kirgansiz (cookie bor). `evil.com` sahifasida yashirin forma `bank.uz/transfer` ga POST yuboradi — brauzer cookie'ni **avtomatik qo'shadi**. Himoya: (1) cookie'larda `SameSite=Lax` yoki `Strict`

(zamonaviy brauzerlarda Lax standart); (2) holatni o'zgartiruvchi amallar faqat POST/PUT/PATCH/DELETE; (3) qo'shimcha himoya uchun CSRF tokeni yoki `Origin` sarlavhasini tekshirish.

Xavfsizlik nazorat ro'yxati (deploy oldidan)

- ☐ HTTPS majburiy, HTTP → HTTPS yo'naltirish
- ☐ `.env` git'da yo'q, production kalitlari kuchli va noyob
- ☐ `NODE_ENV=production`, xato javoblarida stack trace yo'q
- ☐ `helmet`, CORS faqat o'z domeningiz, `express.json({ limit })`
- ☐ Login va ro'yxatdan o'tishda rate limit
- ☐ Barcha kirish ma'lumotlari validatsiya qilinadi; SQL faqat parametrlar bilan
- ☐ Har bir resursda egalik tekshiruvi
- ☐ Cookie: `HttpOnly; Secure; SameSite`
- ☐ `npm audit` toza; Dependabot yoqilgan
- ☐ Bazaning zaxira nusxasi (backup) avtomatik va tiklash sinab ko'rilgan

Unumdorlik (performance)

Qatlam	Usullar
Frontend	Build (minifikatsiya, tree-shaking), kod bo'lish (<code>React.lazy</code>), rasmlarni siqish (WebP/AVIF, <code>loading="lazy"</code> , o'lcham), shriftlarni kamaytirish, CDN
Tarmoq	gzip/brotli, HTTP/2, statik fayllarga uzoq <code>Cache-Control</code> (fayl nomida xesh bilan)
Backend	Event loop'ni bloklamaslik, ulanishlar pool'i, sahifalash, keraksiz ma'lumotni yubormaslik
Baza	Indeksklar, <code>EXPLAIN ANALYZE</code> , N+1 so'rovlardan qochish, kesh (Redis)

O'lchash: Chrome DevTools → **Lighthouse** va **Performance** panellari; asosiy ko'rsatkichlar — **Core Web Vitals**: LCP (asosiy mazmun qancha tez ko'rinadi, < 2.5s), INP (bosishga javob tezligi, < 200ms), CLS (sahifa "sakrashi", < 0.1).



N+1 muammosi

10 ta postni olib, keyin har biri uchun muallifni alohida so'rash = 1 + 10 so'rov. 1000 ta post bo'lsa — 1001 so'rov. Yechim: bitta `JOIN` yoki `WHERE id = ANY($1)` bilan hammasini bir so'rovda olish.

Testlash

Qo'lda tekshirish har safar 10 daqiqa oladi va bir narsani unutasiz. Avtomatik testlar sekundlarda ishlaydi va o'zgarishdan keyin "eski funksiyalar buzilmadimi?" degan savolga ishonchli javob beradi. Ular, shuningdek, kodni qo'rqmasdan qayta yozish (refactoring) imkonini beradi.

Test piramidasi

- **Unit testlar** (ko'p, tez) — alohida funksiyalar: `hisobla(savat)`, reducer, validatsiya sxemasi.
- **Integratsion testlar** (o'rtacha) — bir necha qism birga: API marshruti + baza.
- **E2E testlar** (kam, sekin) — haqiqiy brauzerda foydalanuvchi ssenariysi: Playwright, Cypress.

Yaxshi test **AAA** tuzilishiga ega: **Arrange** (tayyorlash) → **Act** (bajarish) → **Assert** (tekshirish). U amalga oshirishni emas, **xatti-harakatni** tekshiradi.

Amaliy misollar

MISOL 1

Unit testlar: Node.js'ning o'rnatilgan test runner'i

Node 20+ da qo'shimcha kutubxonasiz test yozish mumkin. 4-darsdagi savat funksiyasini test qilamiz.

src/cart.js

```
export const BEPUL_YETKAZISH = 1_000_000;
export const YETKAZISH_NARXI = 25_000;

export function hisobla(items, { promo } = {}) {
  if (!Array.isArray(items)) throw new TypeError('items massiv bo\'lishi kerak');
  const oraliq = items.reduce((s, { narx, soni, chegirma = 0 }) => {
    if (soni < 0 || narx < 0) throw new RangeError('Manfiy qiymat');
    return s + Math.round(narx * (1 - chegirma)) * soni;
  }, 0);
  const promodan = promo === 'BAH0R10' ? Math.round(oraliq * 0.9) :
oraliq;
  const yetkazish = promodan === 0 || promodan ≥ BEPUL_YETKAZISH ? 0 :
YETKAZISH_NARXI;
  return { oraliq, yetkazish, jami: promodan + yetkazish };
}
```

```

import { describe, it } from 'node:test';
import assert from 'node:assert/strict';
import { hisobla, YETKAZISH_NARXI } from '../src/cart.js';

describe('hisobla()', () => {
  it('bo\'sh savat – hammasi nol', () => {
    assert.deepEqual(hisobla([]), { oraliq: 0, yetkazish: 0, jami: 0 });
  });

  it('kichik xaridga yetkazish narxi qo\'shiladi', () => {
    // Arrange
    const items = [{ narx: 100_000, soni: 2 }];
    // Act
    const r = hisobla(items);
    // Assert
    assert.equal(r.oralik, 200_000);
    assert.equal(r.yetkazish, YETKAZISH_NARXI);
    assert.equal(r.jami, 225_000);
  });

  it('1 000 000 dan oshsa yetkazish bepul (chegara qiymati)', () => {
    assert.equal(hisobla([{ narx: 1_000_000, soni: 1 }]).yetkazish, 0);
    assert.equal(hisobla([{ narx: 999_999, soni: 1 }]).yetkazish,
YETKAZISH_NARXI);
  });

  it('mahsulot chegirmasi va promokod birga qo\'llanadi', () => {
    const r = hisobla([{ narx: 500_000, soni: 2, chegirma: 0.1 }], {
promo: 'BAHOR10' });
    assert.equal(r.oralik, 900_000);
    assert.equal(r.jami, 810_000 + YETKAZISH_NARXI);
  });

  it(' noto\'g\'ri kirishda xato tashlaydi', () => {
    assert.throws(() => hisobla(null), TypeError);
    assert.throws(() => hisobla([{ narx: 100, soni: -1 }]), /Manfiy/);
  });
});

```

Terminal

```
node --test
# ▶ hisobla()
#   ✓ bo'sh savat — hammasi nol (0.4ms)
#   ✓ kichik xaridga yetkazish narxi qo'shiladi (0.1ms)
#   ✓ 1 000 000 dan oshsa yetkazish bepul (chegara qiymati) (0.1ms)
#   ✓ mahsulot chegirmasi va promokod birga qo'llanadi (0.1ms)
#   ✓ noto'g'ri kirishda xato tashlaydi (0.2ms)
# i tests 5 pass 5 fail 0

node --test --watch # fayl o'zgarganda qayta
node --test --experimental-test-coverage # kod qamrovi
```

E'tibor bering: **chegara qiymatlari** (999 999 va 1 000 000), bo'sh kirish va xato holatlari alohida tekshirildi — xatolar aynan shu yerlarda yashirinadi.

MISOL 2

API integratsion testlari: Supertest + haqiqiy baza

13-darsdagi Notes API'ni test qilamiz. `app.js` va `server.js` alohida bo'lgani uchun (9-dars) testda port band qilinmaydi.

Terminal

```
npm install -D supertest
npm pkg set scripts.test="node --env-file=.env.test --test"
```

```

import { describe, it, before, after, beforeEach } from 'node:test';
import assert from 'node:assert/strict';
import request from 'supertest';
import { app } from '../src/app.js';
import { pool } from '../src/db.js';

// Har bir test toza bazada ishlaydi
beforeEach(() => pool.query('TRUNCATE users, notes CASCADE'));
after(() => pool.end());

// Yordamchi: ro'yxatdan o'tib, cookie'li "agent" qaytaradi (brauzerga
o'xshab cookie saqlaydi)
async function signup(email = 'ali@test.uz') {
  const agent = request.agent(app);
  await agent.post('/api/auth/register')
    .send({ name: 'Ali', email, password: 'parol12345' })
    .expect(201);
  return agent;
}

describe('Auth', () => {
  it('ro'yxatdan o'tish httpOnly cookie qaytaradi', async () => {
    const res = await request(app).post('/api/auth/register')
      .send({ name: 'Vali', email: 'vali@test.uz', password: 'parol12345'
    })
      .expect(201);
    assert.equal(res.body.user.email, 'vali@test.uz');
    assert.equal(res.body.user.password_hash, undefined); // xesh
    hech qachon qaytmasin!
    assert.match(res.headers['set-cookie'][0], /HttpOnly/);
  });

  it('takroriy email - 409', async () => {
    await signup('a@test.uz');
    await request(app).post('/api/auth/register')
      .send({ name: 'Boshqa', email: 'A@TEST.UZ', password: 'parol12345'
    }) // katta harf ham!
      .expect(409);
  });

  it('noto'lg'ri parol - 401 va umumiy xabar', async () => {
    await signup('b@test.uz');

```

```

const res = await request(app).post('/api/auth/login')
  .send({ email: 'b@test.uz', password: 'xato-parol' })
  .expect(401);
assert.equal(res.body.error.message, 'Email yoki parol noto\'g\'ri');
});

describe('Notes', () => {
  it('cookie\'siz - 401', async () => {
    await request(app).get('/api/notes').expect(401);
  });

  it('CRUD to\'liq sikli', async () => {
    const agent = await signup();

    const { body: created } = await agent.post('/api/notes')
      .send({ title: 'Test', body: 'Matn', tags: ['Ish', 'ish'] })
      .expect(201);
    assert.deepEqual(created.tags, ['ish']); // kichik harf + takrorsiz

    const { body: list } = await agent.get('/api/notes?q=matn').expect(200);
    assert.equal(list.data.length, 1);

    const { body: updated } = await agent.patch(`/api/notes/${created.id}`)
      .send({ pinned: true }).expect(200);
    assert.equal(updated.pinned, true);

    await agent.delete(`/api/notes/${created.id}`).expect(204);
    await agent.get('/api/notes').expect(200).then(r =>
      assert.equal(r.body.data.length, 0));
  });

  it('🔒 boshqa foydalanuvchining eslatmasini o\'zgartirib bo\'lmaydi (IDOR)', async () => {
    const ali = await signup('ali@test.uz');
    const hacker = await signup('hacker@test.uz');
    const { body: note } = await ali.post('/api/notes').send({ title: 'Maxfiy' }).expect(201);

    await hacker.patch(`/api/notes/${note.id}`).send({ title: 'Buzildi' }).expect(404);
  });
});

```

```

    await hacker.delete(`/api/notes/${note.id}`).expect(404);
    const { body } = await hacker.get('/api/notes').expect(200);
    assert.equal(body.data.length, 0);
  });

  it('noma\'lum maydon (user_id) qabul qilinmaydi', async () => {
    const agent = await signup();
    const { body: note } = await agent.post('/api/notes').send({ title:
'X' });
    await agent.patch(`/api/notes/${note.id}`).send({ user_id: '00000000-
0000-0000-0000-000000000000' }).expect(400);
  });

  it('validatsiya: bo\'sh eslatma - 400 va tafsilotlar', async () => {
    const agent = await signup();
    const res = await agent.post('/api/notes').send({ title: '', body: ''
}).expect(400);
    assert.ok(Array.isArray(res.body.error.details));
  });
});

```

Xavfsizlik talablari ham test sifatida yozildi: endi kimdir kelajakda `AND user_id = $2` ni tasodifan o'chirib yuborsa, CI darhol qizarib, xatoni ko'rsatadi.

MISOL 3 React komponentlarini testlash: Vitest + Testing Library

Terminal

```

cd client
npm install -D vitest jsdom @testing-library/react @testing-library/user-
event @testing-library/jest-dom

```

client/vite.config.js (qo'shimcha)

```
export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: './src/test-setup.js',
  },
});
// src/test-setup.js: import '@testing-library/jest-dom/vitest';
```

```

import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { vi } from 'vitest';
import NoteEditor from './NoteEditor';

describe('<NoteEditor />', () => {
  it('bo\'sh eslatmani saqlamaydi va xabar ko\'rsatadi', async () => {
    const onSave = vi.fn();
    render(<NoteEditor onSave={onSave} />);

    await userEvent.click(screen.getByRole('button', { name: 'Saqlash'
}));

    expect(onSave).not.toHaveBeenCalled();
    expect(screen.getByText('Sarlavha yoki matn
yozing')).toBeInTheDocument();
  });

  it('teglarni ajratib, tozalab yuboradi va formani tozalaydi', async ()
=> {
    const onSave = vi.fn().mockResolvedValue();
    const user = userEvent.setup();
    render(<NoteEditor onSave={onSave} />);

    await user.type(screen.getByPlaceholderText('Sarlavha'), 'Xaridlar');
    await user.type(screen.getByPlaceholderText(/Teglar/), ' Uy , bozor,,
');
    await user.click(screen.getByRole('radio', { name: 'green' }));
    await user.click(screen.getByRole('button', { name: 'Saqlash' }));

    expect(onSave).toHaveBeenCalledWith({ title: 'Xaridlar', body: '',
color: 'green', tags: ['uy', 'bozor'] });
    expect(screen.getByPlaceholderText('Sarlavha')).toHaveValue('');
  });

  it('server xatosini ko\'rsatadi', async () => {
    const onSave = vi.fn().mockRejectedValue(new Error('Tarmoq xatosi'));
    render(<NoteEditor onSave={onSave} initial={{ title: 'A', body: '',
tags: [] }} />);
    await userEvent.click(screen.getByRole('button', { name: 'Saqlash'
}));
    expect(await screen.findByText('Tarmoq xatosi')).toBeInTheDocument();
  });
});

```

```
});  
});
```

Terminal

```
npx vitest          # watch rejimi  
npx vitest run      # bir marta (CI uchun)
```

Testing Library falsafasi: komponentni foydalanuvchi ko'rgandek test qiling — CSS sinflari yoki ichki state orqali emas, **rol, yozuv va matn** orqali (`getByRole('button', { name: 'Saqlash' })`). Shu tufayli testlar accessibility'ni ham bilvosita tekshiradi.

MISOL 4

Docker Compose bilan deploy va Render'ga joylash

A) Ko'p bosqichli Dockerfile (frontend + backend bitta image'da)

notes-app/Dockerfile

```
# 1-bosqich: React'ni build qilish
FROM node:22-alpine AS client
WORKDIR /app/client
COPY client/package*.json ./
RUN npm ci
COPY client/ ./
RUN npm run build

# 2-bosqich: faqat production bog'liqliklari bilan server
FROM node:22-alpine
ENV NODE_ENV=production
WORKDIR /app/server
COPY server/package*.json ./
RUN npm ci --omit=dev
COPY server/ ./
COPY --from=client /app/client/dist /app/client/dist
# root emas - xavfsizroq
USER node
EXPOSE 3000
HEALTHCHECK CMD wget -q0- http://localhost:3000/api/health || exit 1
CMD ["node", "src/server.js"]
```

notes-app/.dockerignore

```
**/node_modules
**/.env
**/dist
.git
```

```
services:
  db:
    image: postgres:17
    restart: unless-stopped
    environment:
      POSTGRES_DB: notes
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./server/db/schema.sql:/docker-entrypoint-initdb.d/schema.sql:ro
# birinchi ishga tushishda
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 10

  app:
    build: .
    restart: unless-stopped
    depends_on:
      db:
        condition: service_healthy
    environment:
      DATABASE_URL: postgres://postgres:${DB_PASSWORD}@db:5432/notes #
      "db" - servis nomi = host
      JWT_SECRET: ${JWT_SECRET}
      PORT: 3000
    ports:
      - "3000:3000"

volumes:
  pgdata:
```

Terminal

```
# .env (compose uchun): DB_PASSWORD=... JWT_SECRET=...
docker compose up -d --build
docker compose logs -f app
docker compose exec db psql -U postgres notes -c "SELECT count(*) FROM
notes;"

# Zaxira nusxa
docker compose exec db pg_dump -U postgres notes > backup-$(date +%F).sql
```

VPS'da shu buyruqlarni ishga tushirib, oldiga Nginx + certbot qo'ysangiz — ilova tayyor.

B) Render.com'ga joylash (PaaS — eng oson yo'l)

1. Loyihani GitHub'ga push qiling (7-dars).
2. Render'da **New → PostgreSQL** — bepul baza yarating, "Internal Database URL" ni nusxalang. `schema.sql` ni `psql` orqali bir marta ishga tushiring.
3. **New → Web Service** → repozitoriyani tanlang → Runtime: **Docker** (yuqoridagi Dockerfile ishlatiladi).
4. **Environment:** `DATABASE_URL`, `JWT_SECRET` (yangi, kuchli kalit), `NODE_ENV=production`.
5. **Health Check Path:** `/api/health`. Deploy!
6. Bir necha daqiqadan so'ng ilova `https://notes-xxxx.onrender.com` da HTTPS bilan ishlaydi. Har bir `git push` — avtomatik qayta deploy.
7. O'z domeningiz: Render → Settings → Custom Domain, domen DNS'ida `CNAME` yozuvi (1-dars).



Deploy'dan keyin

Xatolarni kuzatish (Sentry), uptime monitoring (UptimeRobot — sayt o'chsa xabar beradi), loglarni ko'rish va bazaning muntazam zaxira nusxalari — bularsiz production ilova "ko'r" holda ishlaydi.

Mashqlar



Yakuniy topshiriqlar

1. 13-darsdagi loyihani Docker Compose bilan ishga tushiring, so'ng Render (yoki Railway) ga deploy qiling va havolani portfolio'ngizga qo'shing.
2. Loyihaga GitHub Actions CI qo'shing: lint + server testlari + client testlari. Ataylab xato commit qilib, CI qizarishini ko'ring.
3. Lighthouse auditidan o'tkazing va barcha 4 toifada 90+ ballga erishing.
4. OWASP nazorat ro'yxati bo'yicha o'z ilovangizni tekshiring; topilgan har bir muammo uchun test yozing va tuzating.
5. Playwright bilan bitta E2E test yozing: ro'yxatdan o'tish → eslatma yaratish → qidirish → chiqish.



Qo'llanma yakuni

Tabriklaymiz! Siz web qanday ishlashidan boshlab, HTML/CSS/JavaScript, Node.js, Express, PostgreSQL, autentifikatsiya, React, deploy, xavfsizlik va testlashgacha bo'lgan to'liq yo'lni bosib o'tdingiz. Bu — junior full-stack dasturchidan kutiladigan bilimlar poydevori.

Keyingi qadamlar:

- **TypeScript** — katta loyihalarda xatolarni kod yozish paytidayoq ushlash.
- **Next.js** — React uchun full-stack freymvork (server rendering, API marshrutlari).
- **Tailwind CSS** yoki komponent kutubxonalari — tezroq dizayn.
- **Redis, WebSocket, navbatlar** — real vaqt va yuqori yuklama.
- Eng muhimi: **loyihalar qiling**. Har bir yangi loyiha — qo'llanmadagi bilimlarning chuqurlashuvi. 3–5 ta sifatli, deploy qilingan va GitHub'da toza kodli loyiha — eng yaxshi rezyume.

☐ Darsni o'qib bo'ldim